GraduatChildMonitor

Flutter Project – Progress Report

1. Project Overview

This report summarises the completed features, architecture, technologies, and recommendations for the **GraduatChildMonitor** Flutter application as of **May 2026**. The app targets child-monitoring workflows and is built with a clean, scalable architecture following Flutter best practices.

2. Completed Features

Feature	Description
Splash & Onboarding	Introductory screens and first-run setup flow.
Authentication	Login / registration, token handling, and secure storage.
Bottom Navigation	Multi-section bottom nav bar for all primary app screens.
Home Screen	Dashboard and main content feed.
Articles	Listing, filtering, and article detail pages.
Chat	Basic message model and presentation components.
Profile	User profile view and edit flow.
Notifications	Local notification integration and notification list UI.
Today Plan	Daily schedule / plans screen.
Progress Tracking	Progress summary views and tracking components.
Assets & Theming	Custom fonts, color management, and reusable text styles.
Widget Tests	Basic tests under test/ for UI validation.

3. Architecture & Project Structure

The project follows a **Clean Architecture-inspired** layout. Each feature directory contains three layers:

- data/ — repositories, data sources, and model DTOs.
- domain/ — use-cases, entities, and repository interfaces.
- presentation/ — UI widgets, pages, and Cubit state classes.

Key Files

lib/main.dart	App entry point and global initialisation.
lib/core/di/service_locator.dart	Dependency registration (get_it).
lib/core/network/api_client.dart	API client interfaces.
lib/core/network/api_client.g.dart	Generated client implementations.
lib/core/network/token_storage.dart	Token / session management.
lib/core/helpers/notification_helper.dart	Local notification helper.
lib/generated/assets.dart	Generated asset references.
lib/core/error/exceptions.dart	Error / failure models.

4. Technologies & Libraries

Library / Tool	Purpose
Flutter / Dart	Primary platform and language.
flutter_bloc / bloc	State management via lightweight Cubits.
dio	HTTP networking client.
Retrofit-style gen	Code-generated API client via build_runner.
json_serializable	JSON serialisation / deserialisation for models.
get_it	Service-locator dependency injection.
build_runner	Code generation runner for .g.dart files.
flutter_gen	Typed asset reference generation.
flutter_local_notifications	Local push-notification support.

5. UI, Theming & Animations

- Responsive mobile UI with onboarding and bottom navigation.
- Custom typography via **app_text_styles.dart** and **font_manager.dart**.
- Centralised color palette in **color_manager.dart**.
- Lightweight animations through **animation_wrapper** and **staggered_animation_wrapper** components.
- Multi-resolution image assets organised under *assets/images/*.

6. Remaining Work (High Priority)

Tests

Expand unit tests for domain use-cases and repositories. Add widget tests for critical UI components. Integrate end-to-end tests for auth, onboarding, and data synchronisation flows.

Performance

Profile with Flutter DevTools. Optimise list views with ListView.builder and caching. Reduce unnecessary rebuilds. Defer heavy computations to isolates. Adopt cached_network_image for remote images.

Theming & UX Polish

Centralise all theme definitions. Add dark mode and theme variants. Ensure consistent spacing and typography. Standardise component styles (buttons, cards, inputs).

7. Recommendations & Next Steps

API Documentation	Add OpenAPI / Swagger spec or a simple markdown API reference to document all endpoints consumed by api_client.
Test Coverage	Target unit tests for all domain use-cases, widget tests for critical UI, and integration tests for end-to-end flows.
CI Pipeline	Set up a pipeline running flutter analyze, flutter test, and build steps on every pull request.
Image Optimisation	Serve smaller thumbnails for list views; use cached_network_image for network images.
Localisation	Consider flutter_localizations if the app will target multiple languages or regions.
Build Runner	Always regenerate .g.dart files after model changes: flutter pub run build_runner build --delete-conflicting-outputs

8. Setup & Build Notes

Generated files (**.g.dart**) must be regenerated after any model or annotation change. Run the following command from the project root:

```
flutter pub run build_runner build --delete-conflicting-outputs
```

Verify **pubspec.yaml** dependency versions before configuring CI/CD to ensure reproducible builds.